Bagging y bosques aleatorios

Sesión 13 · Módulo 4: Métodos de ensamble

Bootstrap aggregation · Bagging · Bosques aleatorios
Error out-of-bag · Importancia de variables · Dependencia parcial
Datos simulados · scikit-learn

Plan de la sesión (≈90 min)

Qué veremos

- **A.** Panorama: tres formas de combinar modelos (bagging, boosting, stacking)
- **B. Bagging**: bootstrap, reducción de varianza
- **C. Random Forests**: aleatoriedad de variables, error OOB, importancias y dependencia parcial

Una sola idea de fondo: un solo árbol es inestable (visto en sesiones anteriores); bagging y Random Forest combinan **muchos** árboles **en paralelo** para obtener un modelo más estable — la próxima sesión veremos cómo el boosting combina modelos **en serie**, de forma muy distinta.

A1 · Tres formas de combinar modelos

Bagging

En paralelo. Entrena B modelos **independientes** sobre remuestreos distintos y **promedia**. Reduce **varianza**.

Ej.: Random Forest

Boosting

En serie. Cada modelo nuevo corrige los **errores** del anterior; se suman con pesos. Reduce **sesgo** (y algo de varianza).

Ej.: AdaBoost, Gradient Boosting, XGBoost

Stacking

Entrena modelos **distintos** (árbol, SVM, kNN, . . .) y un **meta-modelo** aprende a combinar sus predicciones.

No lo veremos hoy a fondo

Hoy nos enfocamos en las dos familias más usadas en la práctica: **bagging/Random Forests** y **boosting** (AdaBoost, Gradient Boosting, XGBoost).

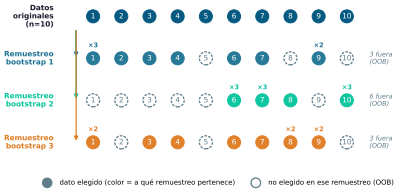
B1 · Bootstrap: remuestrear con reemplazo

La idea

Dado un conjunto de n datos, un **remuestreo bootstrap** toma n datos **al azar, con reemplazo**: algunos aparecen varias veces, otros no aparecen.

Dato clave

En promedio, cada remuestreo deja fuera $\approx \mathbf{36.8}\%$ de los datos originales (ese $\approx 1/e$ es *out-of-bag*, OOB) — lo veremos en C2.



B2 · *Bootstrap aggregating* (bagging): la receta

El algoritmo

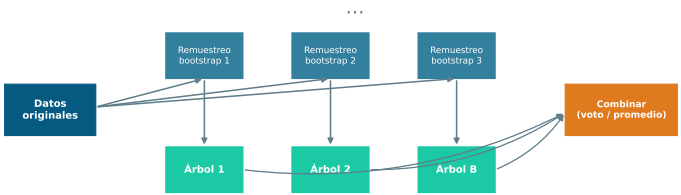
1. Generar B remuestreos bootstrap del entrenamiento.
2. Entrenar un modelo **sin podar demasiado** en cada remuestreo (típicamente árboles, casi sin restringir profundidad).
3. **Combinar** las B predicciones: voto mayoritario (clasificación) o promedio (regresión).

Por qué reduce varianza

Cada modelo individual tiene varianza alta, pero sus errores no están perfectamente correlacionados. Al promediar, los errores **se cancelan parcialmente**: $\mathrm{Var}(\bar{X}) = \frac{\sigma^2}{B}$ si fueran independientes.

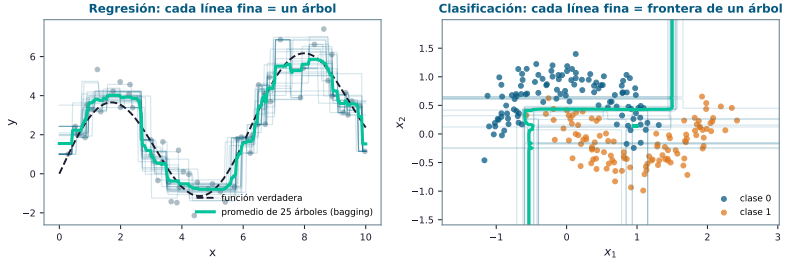
El sesgo de cada modelo individual **no cambia** al promediar — bagging ataca solo la varianza.

B3 · Ilustrando el proceso de bagging



Cada remuestreo bootstrap entrena un modelo distinto (típicamente un árbol poco podado); al final se combinan las B predicciones por voto o promedio.

B3b · Bagging reduce la varianza



Cada línea/frontera delgada es un árbol entrenado en un remuestreo distinto: individualmente son inestables (varianza alta). Al promediar (regresión) o votar (clasificación), las curva/frontera combinada (en verde) es mucho más suave y estable — el sesgo casi no cambia, pero la varianza baja notablemente.

B4 · En scikit-learn: `BaggingClassifier` y `BaggingRegressor`

```python
from sklearn.ensemble import (
    BaggingClassifier, BaggingRegressor)
from sklearn.tree import DecisionTreeClassifier

bagging = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=100,      # cuantos modelos
    max_samples=1.0,       # tamanio de c/remuestreo
    bootstrap=True,        # con reemplazo
    n_jobs=-1, random_state=0)
bagging.fit(Xtr, ytr)
# bagging puede envolver CUALQUIER modelo base,
# no solo arboles (aunque es lo mas comun)
```

Puntos clave

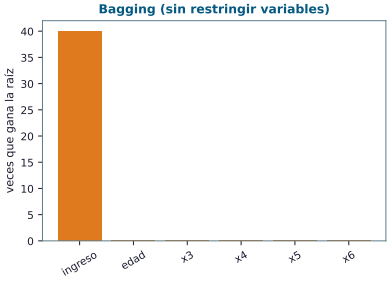
- `estimator` puede ser cualquier modelo base; funciona mejor con modelos de **varianza alta** (árboles sin podar).
- Random Forest (siguiente bloque) es un caso especial de bagging + un ingrediente extra.

C1 · El problema de solo usar bagging

Árboles correlacionados

Si una variable es muy fuerte (como `ingreso` aquí), **casi todos** los árboles bootstrap la usan primero en la raíz: los árboles terminan pareciéndose mucho entre sí.

Promediar árboles **parecidos** reduce poco la varianza: si los errores están correlacionados, $\mathrm{Var}(\bar{X})$ no cae tanto como σ^2/B.



En **40** remuestreos bootstrap sin restringir variables, `ingreso` ganó la raíz en los **40 de 40** árboles.

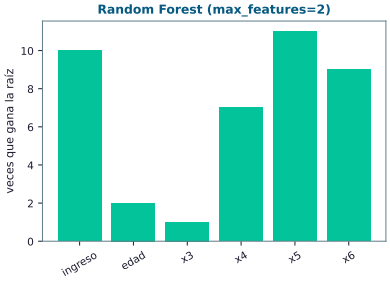
C2 · El ingrediente extra: variables aleatorias en cada corte

La solución de Random Forest

En **cada corte**, en vez de considerar las p variables disponibles, el árbol solo puede elegir entre un subconjunto **aleatorio** de `max_features` variables (típico: $\sqrt{p}$ en clasificación, $p/3$ en regresión).

Dos fuentes de aleatoriedad

(1) **Qué datos** ve cada árbol (bootstrap, como en bagging) y (2) **qué variables** puede usar en cada corte (nuevo).



Con `max_features=2` (de 6 variables), `ingreso` ya solo gana la raíz en **10 de 40** árboles: menos correlación, más diversidad para promediar.

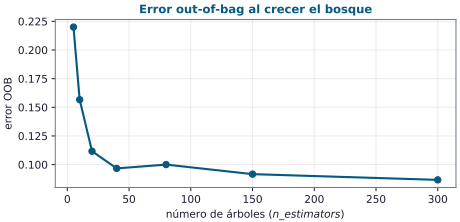
C3 · Error *out-of-bag* (OOB): validación "gratis"

La idea

Cada árbol del bosque deja fuera $\approx 36.8\%$ de los datos (los **OOB** de ese árbol, visto en B1). Para cada dato, se puede predecir usando **solo** los árboles que no lo vieron en entrenamiento.

Por qué importa

El error OOB es casi idéntico al de validación cruzada, pero se obtiene **sin apartar** un conjunto de validación ni reentrenar nada extra.



Con pocos árboles (5) el error OOB es alto y ruidoso (≈22 %); con 300 árboles se estabiliza cerca de su valor final (≈8.7 %).

C4 · En scikit-learn: RandomForestClassifier / Regressor

```python
from sklearn.ensemble import (
    RandomForestClassifier,
    RandomForestRegressor)

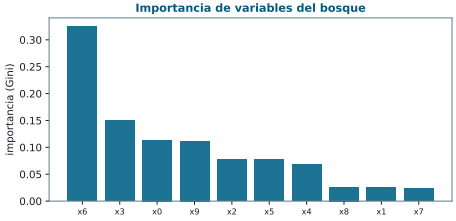
bosque = RandomForestClassifier(
    n_estimators=300,        # cuantos arboles
    max_depth=None,          # casi sin podar
    max_features="sqrt",     # variables por corte
    oob_score=True,          # validacion "gratis"
    random_state=0, n_jobs=-1)
bosque.fit(Xtr, ytr)

bosque.oob_score_              # ~ exactitud de CV
bosque.feature_importances_    # ver C5
```

Puntos clave

- Más árboles (`n_estimators`) casi nunca sobreajusta más; solo cuesta más tiempo.
- Interfaz idéntica a un árbol solo: `fit`, `predict`, `predict_proba`.

C5 · `feature_importances_`: ¿qué variable pesa más?

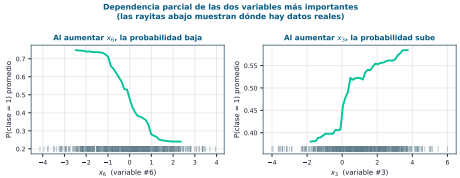


Cómo se calcula

Para cada variable, se suma **cuánto** reduce la impureza (Gini/MSE) en todos los cortes donde se usó, en todos los árboles del bosque, y se normaliza a que sume 1.

Es más **estable** que la importancia de un solo árbol (C1), porque promedia sobre B árboles entrenados con datos y variables distintos.

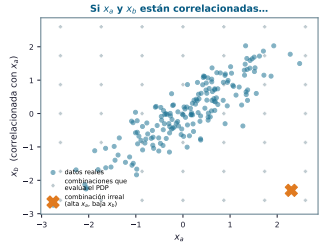
C6 · Dependencia parcial (*partial dependence*)



Las rayitas abajo de cada curva marcan dónde **sí** hay datos reales; donde no hay rayitas, el bosque casi no vio ejemplos ahí y la curva es menos confiable.

Qué responde

"Si **solo** cambiara esta variable (dejando las demás como están, promediando sobre todos los datos), ¿cómo cambiaría la predicción?"



Cuidado: supone variables independientes

Izquierda: los datos reales (azul) solo ocurren en la diagonal, porque x_a y x_b están correlacionadas. El PDP **igual evalúa** toda la rejilla (cruces grises), incluida la "$\times$" roja — una combinación que **nunca** aparece en los datos. Ahí la curva es extrapolación, no algo observado.

C7 · Random Forests: qué se gana y qué se pierde

Se gana

- Mucha menos **varianza** que un árbol solo, con poco ajuste de hiperparámetros.
- `feature_importances_` estable; error OOB sin conjunto de validación aparte.

Se pierde

- Ya **no se puede leer** como reglas si–entonces: se pierde interpretabilidad directa.
- Más lento de entrenar/predecir; sigue sin extrapolar bien fuera del rango de entrenamiento.

En una frase

Un bosque aleatorio cambia interpretabilidad por estabilidad: sigue siendo árboles de decisión por dentro, solo que **muchos**, **diversos** y **promediados**. Bagging y Random Forest reducen **varianza**; no corrigen el **sesgo** de un modelo débil. Para eso está el boosting.

Actividad de laboratorio (individual)

Bagging y Random Forest sobre un mismo conjunto de datos

Cuaderno de Jupyter con un conjunto sintético de clasificación. Trabaja **de forma individual**.

1. Entrena un `BaggingClassifier` y compáralo contra un árbol solo.
2. Compara qué tan seguido una variable domina la raíz, con y sin restringir `max_features`.
3. Grafica el error OOB del bosque contra `n_estimators` y elige un número razonable de árboles.
4. Extrae `feature_importances_` e interpreta cuáles variables pesan más.
5. Compara árbol solo, bagging y Random Forest en una tabla final de exactitud y tiempo de entrenamiento.

Entregable: el cuaderno con la curva de error OOB, las importancias de variable, la tabla comparativa final y tu interpretación de cuál modelo elegirías y por qué.

Resumen

Las ideas centrales de hoy

- **Bagging**: promediar modelos entrenados en remuestreos bootstrap **independientes** $\Rightarrow$ reduce **varianza**.
- **Random Forest**: bagging + variables aleatorias por corte $\Rightarrow$ árboles menos correlacionados, menor varianza todavía; da OOB e importancias "gratis".
- Bagging y Random Forest **no corrigen el sesgo** de un modelo débil — solo reducen varianza combinando modelos **en paralelo**.

En la práctica: Random Forest es casi siempre un buen punto de partida rápido, con poco ajuste de hiperparámetros. En la próxima sesión veremos **boosting** (AdaBoost, Gradient Boosting, XGBoost): la otra gran familia de ensambles, que combina modelos **en serie** para atacar el sesgo.



¿Preguntas?

¡Pasemos a la actividad de laboratorio: bagging y Random Forest!
